GigGuard

Innovation Features & Implementation Plan

Executive Summary

GigGuard is an AI-powered parametric income insurance platform for food delivery workers (Zomato, Swiggy) in India. When external triggers like heavy rain, severe air quality, or floods prevent workers from earning, GigGuard pays lost income automatically — no claim forms, no human review for clean claims, direct UPI payout within minutes.

Beyond the core parametric engine, this document describes six advanced innovations borrowed from the world's leading technology companies — each translated into GigGuard's specific context, with the exact tech stack, schema changes, and implementation timeline for each.

#	Innovation	Source	Expected impact
1	**H3 Hexagonal Geospatial Indexing**	Uber H3	~40% reduction in over-payout from imprecise zone matching
2	**Contextual Bandits for Policy Recommendation**	Netflix personalization	~25% lift in policy purchase conversion
3	**Reinforcement Learning Premium Engine**	Uber TD3 surge pricing	Self-tuning premium; eliminates manual multiplier calibration
4	**Graph Neural Network Fraud Detection**	Stripe Radar + fintech GNN	3–5× more fraud ring detection vs. Isolation Forest
5	**Causal Inference Claim Validation**	Netflix/Spotify causal ML	~30% reduction in false-positive claim payouts
6	**Smart Contract Auto-Execution**	AXA Fizzy blockchain	Zero human claims processing; on-chain policy transparency

Innovation 1 · Borrowed from Uber H3

Hexagonal Geospatial Indexing

What the original technique does

Uber's H3 system divides the entire Earth into a hierarchical grid of hexagonal cells. At resolution 8 — which Uber uses for its city-level demand modeling — each hexagon covers approximately 0.74 km². Because hexagons tile perfectly with no gaps and have equal distance from center to every edge, they eliminate the directional bias that rectangular grids introduce. Each cell has a unique 64-bit integer ID that encodes its position, resolution, and parents/children in the hierarchy. Uber uses H3 to localize surge pricing, driver positioning, and demand prediction to sub-kilometer precision — something impossible with vague text zone names like "South Delhi."

Why GigGuard needs it

GigGuard's current schema stores zone as a free-text VARCHAR. This creates the biggest financial risk in any parametric insurance system: basis risk — paying workers who were not actually in the affected area. When OpenWeatherMap reports heavy rain at a GPS coordinate, we currently map it to the entire "Lajpat Nagar zone" — which might span 12 km². Workers at the dry end of that zone get paid for rain they never experienced. H3 eliminates this. A rain event has a precise lat/lng coordinate, which we convert to an H3 hex ID. We then pay workers whose stored home hex falls within that hex's k=1 ring (the hex plus its 6 immediate neighbors — covering roughly a 2 km radius). Nothing outside that ring gets triggered.

Tech stack

Layer	Technology	Notes
Backend (Node.js)	`h3-js@4.x`	Official Uber H3 JS bindings. latLngToCell(), gridDisk(), cellToParent()
ML Service (Python)	`h3@3.7.x`	Python H3 bindings. Same API, used in premium + fraud models
Database	`PostgreSQL + BIGINT`	H3 cell IDs are 64-bit integers. Store as BIGINT, not VARCHAR
Frontend	`h3-js (browser build)`	Display hex boundaries on the worker dashboard map (Leaflet.js)
Weather trigger	`OpenWeatherMap API`	lat/lng from API → latLngToCell(lat, lng, 8) → query affected workers

Schema changes

Table	Column change	Type	Details
`workers`	`zone (remove)`	VARCHAR → deprecated	Kept for 1 sprint for backward compatibility, then dropped
`workers`	`home_hex_id (add)`	BIGINT NOT NULL	H3 res-8 cell ID of worker's registered home address
`workers`	`active_hex_id (add)`	BIGINT NULLABLE	Live delivery hex, updated every 30 min via platform webhook
`disruption_events`	`affected_hex_ids (add)`	BIGINT[] (array)	All hex IDs in the event's k-ring. Indexed with GIN.
`policies`	`zone (remove)`	VARCHAR → deprecated	Policy now references worker's home_hex_id at purchase time

Core code change (trigger monitor)

```
// BEFORE (imprecise text zone matching)
const workers = await db.query(`SELECT * FROM workers WHERE city=$1 AND zone=$2`, [city, zone]);

// AFTER (H3 precision matching)
const eventHex = h3.latLngToCell(lat, lng, 8);
const ringHexes = h3.gridDisk(eventHex, 1);  // hex + 6 neighbors
const workers = await db.query(
  `SELECT * FROM workers WHERE home_hex_id = ANY($1::bigint[])`,
  [ringHexes]
);
```

Implementation timeline

Phase 2 — Week 3 March 21–28, 2026

- Day 1: npm install h3-js, pip install h3. Add home_hex_id BIGINT to workers table.
- Day 2: Migration script — geocode existing city+zone strings to H3 hex IDs via Google Maps API (one-time backfill).
- Day 3: Update trigger monitor — replace zone string matching with gridDisk() query.
- Day 4: Add GIN index on disruption_events.affected_hex_ids.
- Day 5: Test with simulated rain event — verify only k=1 ring workers are triggered, not entire city.

~40%	0.74 km²	k=1 ring	1 day
reduction in basis risk over-payout	avg hex area at resolution 8	coverage radius ≈ 2 km	migration effort (one-time backfill)

Innovation 2 · Borrowed from Netflix

Contextual Bandits for Policy Recommendation

What the original technique does

Netflix personalizes the artwork thumbnail shown for each title using contextual bandits — specifically, a Thompson Sampling algorithm. Each thumbnail variant is an "arm." The context is the user's viewing history, time of day, and device. The bandit continuously explores which arm maximizes play-rate for each context cluster, balancing exploration (trying new variants) vs. exploitation (showing the known best). This eliminated the need for manual A/B test scheduling — the bandit runs 24/7, updating its belief about each arm's expected reward after every impression.

Why GigGuard needs it

GigGuard currently offers the same policy to every worker: ₹35 base rate, same coverage tiers. But a Swiggy worker in Delhi during monsoon has a completely different risk appetite than a Zomato worker in Chennai in February. A contextual bandit learns which coverage tier (₹29 / ₹44 / ₹65 / ₹89) each worker segment is most likely to purchase — and surfaces that tier first in the onboarding UI. Over time, it discovers non-obvious correlations: maybe experienced workers (avg_daily_hours > 8) strongly prefer higher coverage, while new workers are price-sensitive. No analyst has to define these segments manually.

Tech stack

Layer	Technology	Notes
ML Service (Python)	`scikit-learn + numpy`	Thompson Sampling bandit — pure numpy, no heavy framework needed
State persistence	`PostgreSQL JSONB`	Bandit belief state (alpha/beta per arm per context) stored in DB
Backend API	`Express.js`	POST /api/recommend-tier — returns arm for worker's context
Frontend	`Next.js 14`	Onboarding Step 2 shows recommended tier first, others below
Reward signal	`PostgreSQL events`	Policy purchased = reward 1. Session ended without purchase = reward 0

Algorithm design

Thompson Sampling is the algorithm of choice because it handles cold-start gracefully (uniform Beta(1,1) prior means all arms are equally likely until data accumulates) and is trivially parallelizable. Each arm k has a Beta distribution Beta(alpha_k, beta_k). To select an arm, sample from each distribution — the arm with the highest sample wins. After observing a reward r, update: alpha_k += r, beta_k += (1 - r).

> **Context encoding for GigGuard**
> City bucket (5 cities) × Platform (zomato/swiggy) × Season (4 seasons) × Experience tier (new/mid/veteran based on avg_daily_hours)
> = 5 × 2 × 4 × 3 = 120 context buckets, each with 4 arms (coverage tiers)
> = 480 Beta distributions total — trivially small, stored as a single JSONB object in PostgreSQL

Schema changes

Table	New column	Type	Purpose
`workers`	`experience_tier`	VARCHAR(10)	new / mid / veteran — computed from avg_daily_hours at registration
`policies`	`recommended_arm`	INTEGER	Which tier (0–3) the bandit recommended for this worker
`policies`	`arm_accepted`	BOOLEAN	Whether worker chose the recommended arm (reward signal)

(new table)	bandit_state	JSONB	context_key → {arms: [{alpha, beta}, ...]} — one row, updated in-place

Implementation timeline

Phase 2 — Week 4 March 29 – April 4, 2026

- Day 1-2: Build ThompsonSamplingBandit class in Python (ml-service/bandits/policy_bandit.py). Unit tests with mock reward signals.
- Day 3: Add POST /recommend-tier endpoint to Flask service. Returns arm index + expected premium for worker context.
- Day 4: Integrate into Next.js onboarding Step 2 — recommended tier shown with "Recommended for you" badge.
- Day 5: Wire reward signal — on policy purchase, call POST /bandit-update with context + reward.

~25%	480	0	4 days
lift in policy purchase conversion (Netflix baseline)	context-arm pairs tracked automatically	manual A/B tests needed	implementation effort

---

Innovation 3 · Borrowed from Uber / DeepMind

Reinforcement Learning Premium Engine

What the original technique does

Uber uses an off-policy deep RL algorithm (TD3 — Twin Delayed DDPG) for surge pricing. The agent observes a state vector (supply, demand, time, location cluster) and outputs a price multiplier. The reward is a composite: driver earnings + rider completion rate + marketplace efficiency. Crucially, the agent learns that pricing too high collapses demand (no riders), and pricing too low is unprofitable. No human needs to hand-tune the multipliers — the agent discovers the optimal balance through millions of simulated episodes before going live.

Why GigGuard needs it

GigGuard's Phase 1 premium formula is: weekly_premium = 35 × zone_multiplier × weather_multiplier × history_multiplier. These multipliers are hand-crafted. If zone_multiplier for Dharavi is set to 1.4 but the actual claim rate there is 0.8×, GigGuard overcharges workers and loses them to competitors. An RL agent replaces this formula entirely. It learns: "for this state (zone risk, forecast, worker history, competitor pricing), what premium maximizes both purchase rate AND loss ratio below 80%?" The agent self-corrects weekly as new claim data arrives.

Tech stack

Layer	Technology	Notes
RL Framework	`Stable-Baselines3 (PyTorch)`	TD3 or SAC agent. SAC preferred for continuous action spaces.
Environment	`custom Gym env (Python)`	gigguard_env.py — simulates weekly policy market. Step = one week.
State space	`numpy float32[8]`	[zone_risk, rain_prob_7d, aqi_avg_7d, claim_rate_90d, worker_hours, platform_enc, season_enc, competitor_price]
Action space	`Continuous [0.5, 2.0]`	Premium multiplier. Applied on top of ₹35 base rate.
Reward	`Python composite fn`	r = purchase_rate_reward - loss_ratio_penalty - churn_penalty
Model serving	`Flask + PyTorch`	POST /predict-premium — replaces LinearRegression endpoint
Training data	`historical_claims.csv`	Simulate from Phase 2 real data. 10k episodes of synthetic rollouts.

Reward function design

```
def compute_reward(premium, purchased, claim_filed, payout_amount):
    # Worker bought the policy
    purchase_reward = 1.0 if purchased else -0.3

    # Loss ratio penalty — paid out more than 80% of premium is bad
    loss_ratio = payout_amount / max(premium, 1)
    loss_penalty = max(0, loss_ratio - 0.8) * 5.0

    # Churn penalty — premium too high → worker doesn't buy next week
    churn_penalty = 0.5 if (premium > 60 and not purchased) else 0

    return purchase_reward - loss_penalty - churn_penalty
```

Implementation timeline

Phase 3 — Week 5 April 5–11, 2026

- Day 1-2: Build GigGuardEnv (gymnasium.Env subclass). Define state, action, reward. Unit-test with random agent.
- Day 3-4: Train SAC agent on 50k simulated episodes. Evaluate: does it converge to reasonable premiums (₹30–₹80)?
- Day 5: Replace LinearRegression in /predict-premium endpoint with trained SAC policy. A/B shadow test vs. formula.
- Day 6-7: Evaluate on held-out validation set — compare loss ratio vs. baseline formula. Target < 75% loss ratio.

Self-tuning	**< 75%**	**Weekly**	**7 days**

premiums — no manual multiplier calibration	target loss ratio (vs 80% baseline)	model update cycle from new claim data	implementation effort

---

Innovation 4 · Borrowed from Stripe Radar

Graph Neural Network Fraud Detection

What the original technique does

Stripe's Radar fraud system builds a dynamic heterogeneous graph connecting cards, devices, merchants, email addresses, and IP ranges. Even if a single fraudulent account looks clean in isolation, a Graph Neural Network (GNN) flags it because it shares graph edges with known-bad entities. The GNN aggregates neighborhood information — "your neighbors are suspicious, therefore your risk score is elevated" — which is fundamentally impossible for any model that looks at one account at a time.

Why GigGuard needs it

GigGuard's Phase 1 fraud scorer is Isolation Forest — a model that evaluates each worker's claim independently. This misses the most common real-world fraud pattern: a ring of 10–20 workers who all claim the same "disruption event" from the same neighborhood, all registered from the same device, and all receiving payouts to the same UPI VPA. Isolation Forest sees 10 individually-normal claims. A GNN sees the ring immediately — because the graph edge "shares_upi_with" creates a dense cluster of high-risk nodes.

Tech stack

Layer	Technology	Notes
GNN Framework	`PyTorch Geometric (PyG)`	GraphSAGE model. pip install torch-geometric. Works on CPU for our scale.
Graph construction	`NetworkX + SQLAlchemy`	Build heterogeneous graph from DB at training time. Incremental update at inference.
Node types	`4 types (see below)`	workers, claims, disruption_events, upi_addresses — heterogeneous graph
Model serving	`Flask`	POST /score-fraud accepts claim_id, returns fraud_score + graph_flags
Fallback	`Isolation Forest (keep)`	If GNN service unavailable, fall back to existing model. Zero downtime.
Monitoring	`MLflow`	Track fraud_score distribution. Alert if mean score spikes — may indicate model drift.

Graph design

Node type	Features	Key edges

worker	risk_score, platform, city, hex_id, claim_frequency_30d	filed_claim → claim node
claim	trigger_type, payout_amount, fraud_score, disruption_hours	part_of → disruption_event, paid_to → upi_address
disruption_event	trigger_type, affected_count, total_payout, severity	(source of claim edges)
upi_address	registration_date, total_payouts_received, worker_count	receives_payout ← claim

The fraud ring signal
A ring of 15 workers all connected to the same upi_address node creates a star-shaped subgraph.
GraphSAGE aggregates neighbor features: the upi_address node's "worker_count = 15" feature propagates back to each worker node.
The model learns: worker_count > 5 on a UPI node = high fraud probability, regardless of individual worker features.

Implementation timeline

Phase 3 — Week 5–6 April 5–17, 2026

- Week 5, Day 1-3: Build graph construction script. Pull workers+claims+payouts from DB. Build NetworkX hetero graph. Verify edge counts.
- Week 5, Day 4-5: Train GraphSAGE on synthetic fraud ring data (generate 100 rings of 5–20 workers pointing to same UPI).
- Week 6, Day 1-2: Evaluate precision/recall on held-out rings. Target: recall > 0.90 on rings of size >= 5.
- Week 6, Day 3: Replace /score-fraud endpoint with GNN scorer. Keep Isolation Forest as fallback.
- Week 6, Day 4: Add graph_flags field to claims table — JSON list of flagged graph patterns (e.g. "shared_upi_with_5_workers").

3–5×
better fraud ring detection vs. Isolation Forest

>90%
target recall on coordinated fraud rings

4 node types
in heterogeneous graph

0 downtime
IsolationForest fallback always available

Innovation 5 · Borrowed from Netflix / Spotify

Causal Inference Claim Validation

What the original technique does

Netflix uses difference-in-differences (DiD) and doubly-robust estimators to separate "the recommendation caused this watch" from "the user was going to watch anyway." Without causal inference, Netflix would over-credit its recommendation system — seeing correlation and calling it causation. Spotify applies the same logic to playlist performance: did featuring a song cause streams, or did popular songs get featured? The academic foundation is the Rubin Potential Outcomes framework — every observed outcome has a counterfactual ("what would have happened otherwise?").

Why GigGuard needs it

The most expensive false positive in parametric insurance is paying a worker who was already offline before the trigger event. Consider: a worker whose phone died at 3pm, who then claims a 4pm rainstorm caused their income loss. GigGuard's current system pays based on trigger alone — no counterfactual check. Causal inference adds one question before every payout: "Would this worker have been working if the trigger hadn't happened?" We estimate this using their historical activity on the same weekday at the same time — the difference-in-differences counterfactual. Only the causal income loss gets paid.

Tech stack

Layer	Technology	Notes
Causal library	`DoWhy + econml (Python)`	Microsoft's causal ML libraries. pip install dowhy econml. Industry standard.
Counterfactual data	`PostgreSQL (workers table)`	Historical active_hours per weekday per time-of-day — stored as JSONB baseline
Estimator	`Difference-in-Differences`	Compare disruption day activity vs. same-weekday-3-week average at same hour
Platform integration	`Zomato/Swiggy webhook (Phase 3)`	Real-time order count as proxy for worker activity. Requires partner API access.
Fallback (Phase 2)	`Worker self-report + GPS`	Until platform API available: worker taps "I was working" in app. GPS confirms location.
Claims table	`PostgreSQL`	Add counterfactual_loss_hours DECIMAL(4,2) — only this delta gets paid

The DiD estimator for GigGuard

```
def estimate_causal_loss(worker_id, disruption_start, disruption_hours, db):
    # Hours worker was active on disruption day (treatment period)
    treated_hours = get_platform_activity(worker_id, disruption_start, disruption_hours)

    # Same weekday + time window over last 3 weeks (control)
    control_hours = get_baseline_activity(worker_id, disruption_start, weeks=3)

    # DiD estimate: how many hours did the disruption CAUSE to be lost?
    causal_loss = max(0, control_hours - treated_hours)

    # Store counterfactual for audit trail
    return causal_loss, control_hours
```

Schema changes

Table	New column	Type	Purpose
`workers`	`activity_baseline`	JSONB	{"mon_9am": 0.87, "tue_3pm": 0.92, ...} — hourly activity probability
`claims`	`counterfactual_loss_hours`	DECIMAL(4,2)	Causal estimate of hours lost. Payout = f(this), not trigger disruption_hours
`claims`	`baseline_active_hours`	DECIMAL(4,2)	Control period activity. Stored for audit transparency.
`claims`	`causal_payout_amount`	DECIMAL(10,2)	Revised payout based on causal loss, replacing payout_amount for flagged claims

Implementation timeline

Phase 3 — Week 6 April 11–17, 2026

- Day 1-2: Build activity_baseline population script — compute hourly activity probabilities for all existing workers from historical policies.
- Day 3: Build DiD estimator function. Test: simulated worker offline 2h before rain event → causal_loss_hours reduced by 2h.
- Day 4: Integrate into claim pipeline — after fraud check, run causal estimator. Update causal_payout_amount.
- Day 5: Add causal breakdown to worker payout notification: "Rain caused 3.2h loss. We're paying for 3.2h (not 4h trigger)."

~30%	DiD	Transparent	5 days
reduction in false-positive claim payouts	difference-in-differences estimator	workers see causal breakdown in app	implementation effort (Phase 3)

Innovation 6 · Borrowed from AXA Fizzy

Smart Contract Auto-Execution

What the original technique does

AXA's Fizzy platform (2017) was the insurance industry's first live blockchain parametric product. Flight delay insurance was encoded as an Ethereum smart contract. When the Flightradar24 data oracle confirmed a delay >= 2 hours, the contract self-executed — calling the payout function directly, with no

human in the loop and no claim form. The contract was immutable: AXA literally could not deny a valid claim once deployed. This is the gold standard for trust and transparency in parametric insurance.

Why GigGuard needs it

GigGuard's target users — delivery workers earning ₹600–800/day — have zero reason to trust a new insurance platform. "Trust us, we'll pay automatically" is a marketing claim. An on-chain policy is a mathematical guarantee. The worker's policy terms, UPI address, and payout conditions are encoded in an immutable Solidity contract. When Chainlink's weather oracle confirms rain > 15mm/hr at the worker's H3 hex, the contract calls Razorpay directly. No human can block the payout. Workers can verify their own policy on Etherscan. This is the feature that converts the most skeptical user.

Tech stack

Layer	Technology	Notes
Smart contract	`Solidity 0.8.x`	GigGuardPolicy.sol — deployed to Polygon (low gas fees, ~$0.001/txn)
Blockchain network	`Polygon (MATIC)`	Ethereum-compatible. Gas fees ~100× cheaper than mainnet. 2s block time.
Oracle	`Chainlink Any API`	Chainlink node fetches OpenWeatherMap + AQICN on-demand. Verifiable on-chain.
Development env	`Hardhat + ethers.js`	Local blockchain for testing. npx hardhat node simulates Chainlink oracle.
Payment bridge	`Razorpay Payout API`	Contract calls backend webhook → backend calls Razorpay. UPI payout in <30s.
Frontend	`ethers.js + MetaMask (optional)`	Workers can verify their policy contract address in the app. MetaMask optional.
Regulatory note	`IRDAI sandbox`	Required for live deployment. Hackathon demo uses Polygon Mumbai testnet only.

Core Solidity contract design

```
// GigGuardPolicy.sol (simplified)
contract GigGuardPolicy {
    struct Policy {
        address worker;
        string upiAddress;      // Razorpay UPI VPA
        uint64  hexId;          // H3 hex ID at resolution 8
        uint256 coverageAmount; // in paise (₹1 = 100 paise)
        uint256 weekStart;
        bool    active;
    }

    // Triggered by Chainlink oracle when rain > 15mm at hexId
    function triggerPayout(bytes32 policyId, uint256 triggerValue)
        external onlyOracle {
        Policy storage p = policies[policyId];
        require(p.active, 'Policy not active');
```

```
        require(triggerValue > RAIN_THRESHOLD, 'Threshold not breached');
        p.active = false;  // Prevent double-payout
        emit PayoutTriggered(policyId, p.upiAddress, p.coverageAmount);
        // Backend webhook listens to this event → calls Razorpay
    }
}
```

Implementation timeline

Post-hackathon (Scale phase) May 2026 onward

- Hackathon demo: Deploy GigGuardPolicy.sol to Polygon Mumbai testnet. Demo in 5-min Phase 3 video — show contract on testnet explorer.
- Post-hackathon Month 1: Apply to IRDAI regulatory sandbox (Insurance Regulatory Sandbox, 2020 framework).
- Post-hackathon Month 2-3: Build Chainlink oracle integration. Test with live OpenWeatherMap data.
- Post-hackathon Month 4: Pilot with 100 workers in Mumbai. Policy address shown in app. UPI payout from contract webhook.
- Full launch: Subject to IRDAI sandbox approval. Target Q4 2026.

Regulatory note

Smart contract execution does not replace IRDAI licensing — it is the payout mechanism, not the insurance product itself.

The legal insurance contract remains with GigGuard (or its insurer partner). The blockchain is the transparent execution layer.

IRDAI's 2020 Regulatory Sandbox specifically permits blockchain-based insurance experiments.

Zero	< 30s	Immutable	$0.001
human claims processing for clean claims	contract trigger to UPI payout	policy terms — insurer cannot deny valid claim	gas cost per transaction on Polygon

---

Master Implementation Timeline

All six innovations mapped against the three hackathon phases.

Innovation	Phase 2 (Mar 21–Apr 4)	Phase 3 (Apr 5–17)	Post-hackathon	Key dependency
H3 Indexing	Full implementation	GIN index + analytics	—	h3-js, one-time geocode migration
Contextual Bandits	Full implementation	Model retraining	—	Policy purchase event stream

RL Premium Engine	Shadow test	Full rollout	Production A/B test	Stable-Baselines3, PyTorch
GNN Fraud Detection	Graph schema prep	Full implementation	Continuous retraining	PyTorch Geometric, fraud training data
Causal Inference	—	Full implementation	Platform API (Zomato)	DoWhy, worker baseline activity data
Smart Contract	Demo (testnet)	Demo video feature	IRDAI sandbox apply	Polygon testnet, Solidity 0.8.x